UNIVERSITA' DEGLI STUDI DI MILANO BICOCCA

Analysis of time series with Recurrent Neural Nets

Lecture 15

Course of:
Statistics and data analysis

Federico De Guio - Martino Borsato

Why is it interesting to analyze **time series**?

- Time series analysis is about **understanding the patterns hidden in the data**
  - cyclical patterns, unexpected events, general trends

- It's a powerful tool for **making sense of the past and using that understanding to predict the future**
  - Detection of **recurring patterns** and anomalies over time
  - Identify potential risks, guide decisions

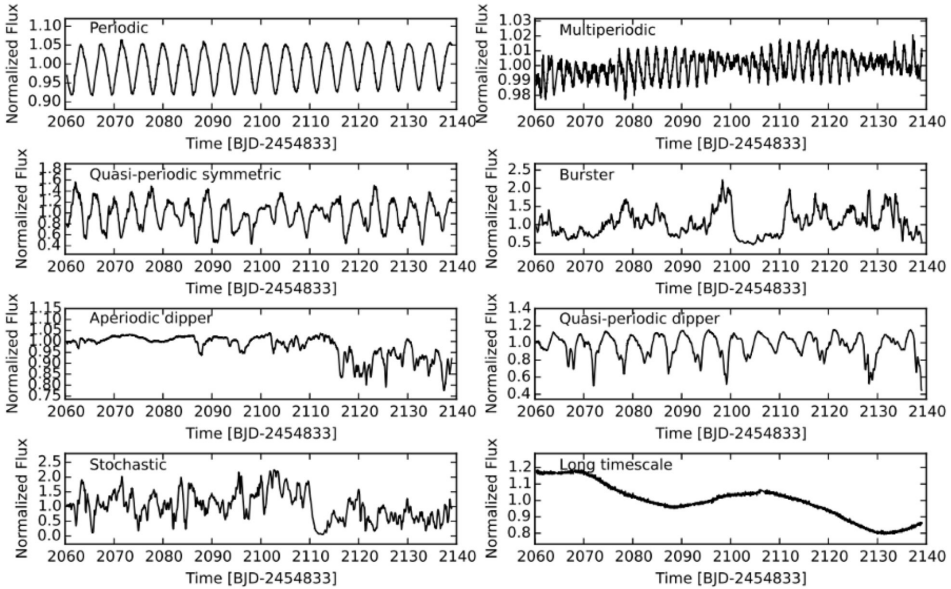
- **Many applications:**
  - Finance, economics, healthcare, environment, …

Time series definition and data tasks

- A time series is a **set of time-tagged measurements**: $\{X_i(t_i)\}$ possibly with observation errors σ_i

- Data points are **not independent and not identically distributed**
  - **Data is sequential** → next point depends on previous point
  - Series affected by **trends** and **drifts**

Astronomical light curves (how the star luminosity changes over time)



Stationarity

- For an algorithm to **effectively model the underlying patterns** in a time series, **stationarity is a desirable condition** to ensure that the learned relationships remain valid over time

- **Strong** stationarity: joint probability distribution is translationally invariant
  - Hard to find in data

- **Weak** stationarity: mean, variance, autocorrelation are constant
  - No trends in the data + amplitude of fluctuations should be constant
  - Relation between one point and the one preceding it, doesn't depend on the absolute time

- When dealing with features characterized by persistent drifts, **restoring stationarity** is a fundamental preprocessing step to ensure model reliability

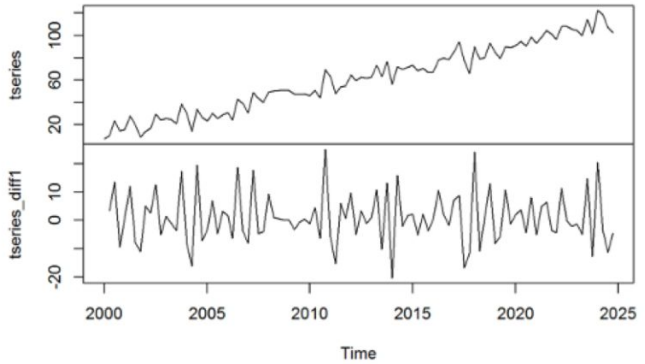
Options to **restore stationarity**

- Transformations to achieve stationarity (constant location and scale)
  - Difference the data: $Z_i = X_i - X_{i-1}$
  - Detrend the data: $Z(t) = X(t) - f(t)$
  - Stabilize the variance:

    $Z(t) = \sqrt{X(t) + A}$ or $\log(X(t) + A)$
- Test with autocorrelation function (ACF):

$$\rho_k = \frac{\text{cov}(X_t, X_{t+k})}{\text{var}(X_t)\text{var}(X_{t+k})}$$

(should be time-independent if stationary)



Autoregressive vs **direct forecasting** models

- **Autoregressive models** use observations from previous time steps as input to predict the value of the same variable at the next time step. The assumption is that **the knowledge of the past is the best predictor of the future**
  - Extreme approach is **recursive forecasting**: use the prediction at step t_i as input to predict t_{i+1}
  - High risk of feedback loop → **bias**

- An alternative approach is **direct forecasting**: predict a precise time frame in the future without going through the intermediate steps

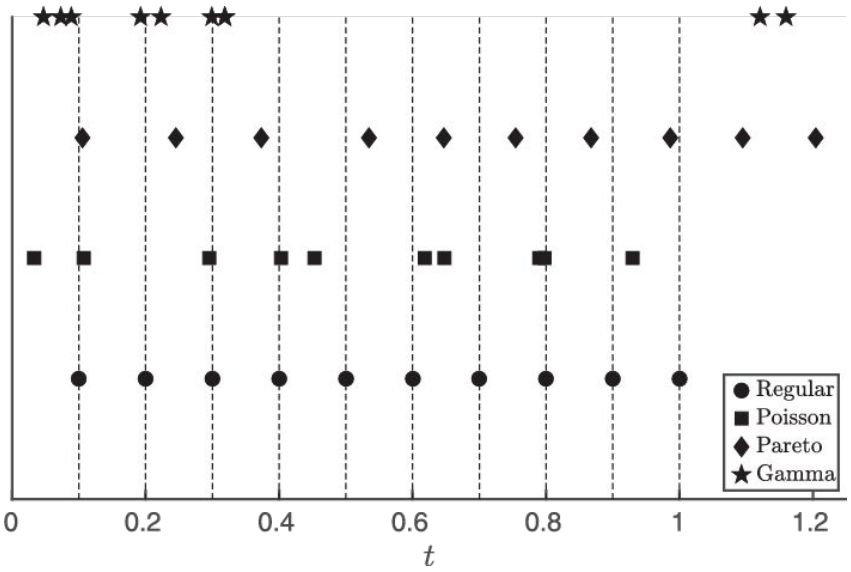
Feature	Recursive Forecasting (Autoregressive)	Direct Forecasting
Logic	y_{t+1} feeds into the prediction for y_{t+2}.	The past points directly to the target y_{t+H}.
Error Accumulation	**Yes:** a small error today compromises all future predictions (snowball effect).	**No:** each prediction is independent of previous errors.
Model Bias	The model is optimized for short-term dynamics (1-step ahead).	The model is optimized for a specific time horizon (e.g., 5-steps ahead).

Dealing with real world data: **sampling**

- **Continuous time process** → observations are a random sample drawn from a continuous process described by some differential equation

- Ideal case: **even or regular sampling**
  - $y(t) = x(t_0 + n\Delta t)$ where $n = 0,1,\ldots, m$

- More common: **uneven or irregular sampling**
  - $y(t) = x(t_0), \ldots, x(t_m)$



- Options for regularization before feeding the algo
  - Bin data onto regular grid:

$$y(t) = \frac{\sum_i w_i x_i}{\sum_i w_i} \text{ for } t_i \in [t_a, t_b]$$

  - Interpolate: linear, spline, Gaussian process

Limits of standard Neural Networks for sequential data

- Not all problems can be handled with fixed-length inputs and outputs
  - Speech recognition or time-series prediction require a system to store and use context information (memory)

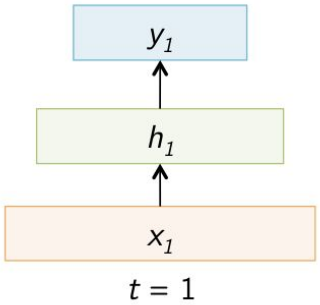
- Example: if I'm analyzing light patterns from stars, observations for a star could be 100d and 1000d for another one → **rigid models don't work!**

- Hard/impossible to choose a fixed context window

- → **read data in sequence and update the status (memory) of the model**
  - Can't work with a fixed picture of the time information
  - Need to iterate over the data with a sliding window approach

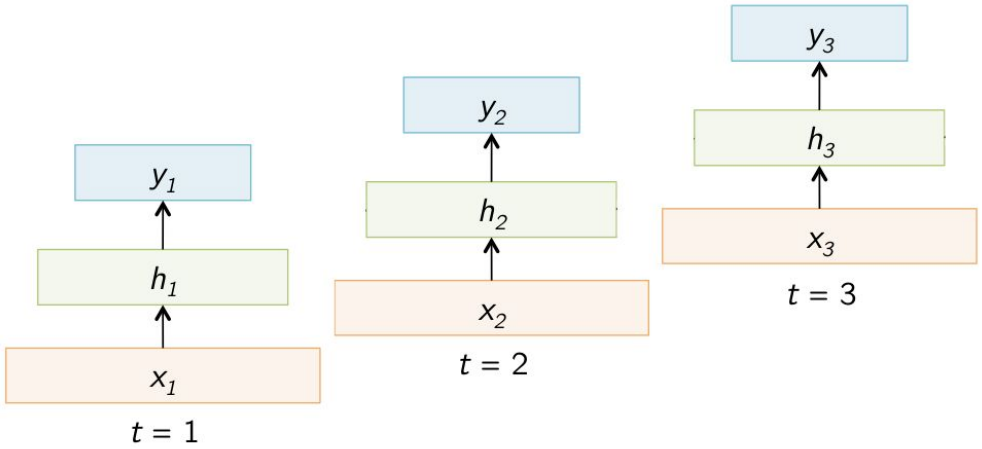
Feed Forward NN

- Classic dense approach



Time-distributed NN

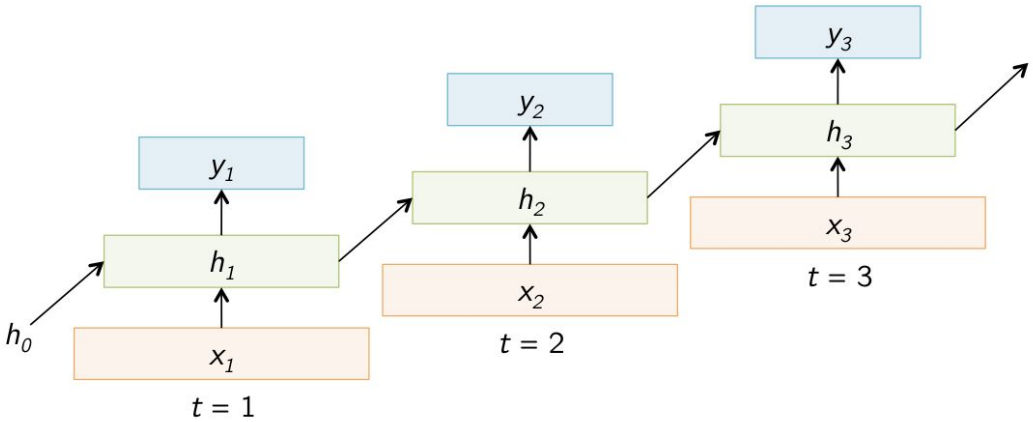
- **Time-distributed** NN shares parameters across time steps
  - Each element of the series is analyzed independently



- One example: extract features from each frame of a video

Recurrent NN

- **Recurrent NN** considers current input **and** previous hidden state → **memory!**
  - Data flows both vertically (independent events) and horizontally (time)



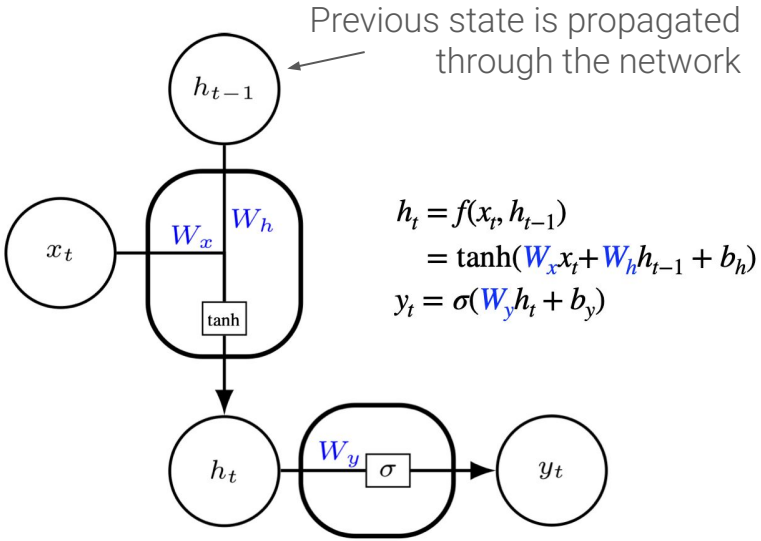
- One example: understand the “movement” of an object in the video

What is a Recurrent Neural Network (**RNN**)?

- RNN are **specific ANN architectures optimised to identify long-term correlations** in sequence of informations of variable lengths
  - natural language processing, signal processing, time series forecasting, etc…

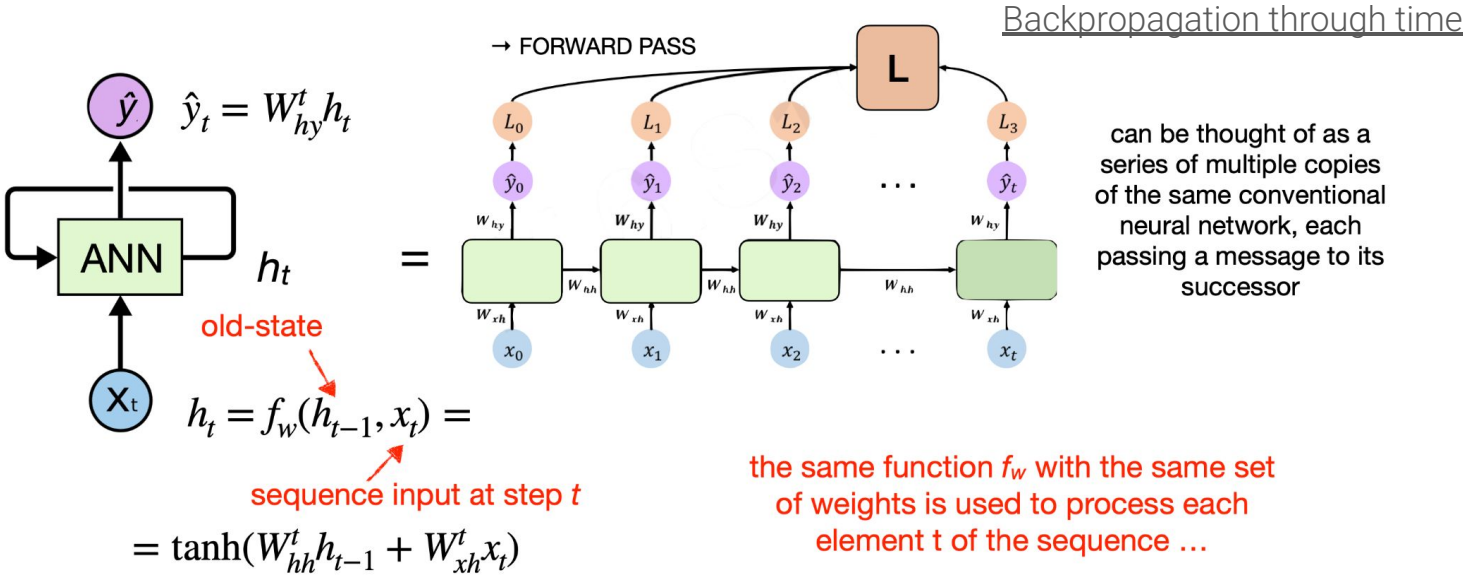
- Typical task for a RNN: **given a sequence of features** (text, music, ...), **predict one or more targets** (the next word on a phrase, the weather in the next 24h, ...)

- A RNN is able to:
  - take in input sequences of variable length
  - keep track of dependencies between elements that are distant in the sequence
  - keep information on the order between the elements of the sequence
  - use shared weights so that identified correlations between elements can be transferred in diverse points of the sequence



RNN implementation

- A RNN processes the input in a loop (**recurrent connection**) that allows the persistency of the informations during the entire processing of the sequence's elements
- **Base module:** is a NN that analyses the t element of the input sequence x_t and produce the output h_t (**hidden state**). h_t is passed to the same network during the processing of the next element of the sequence



Vanishing and **exploding** gradients

- Gradients depend on the balance (weight x activation function derivative)
  - If small weight and small derivative → **vanishing gradient**
  - If large weight → risk of **exploding gradient** → small derivative is to be preferred

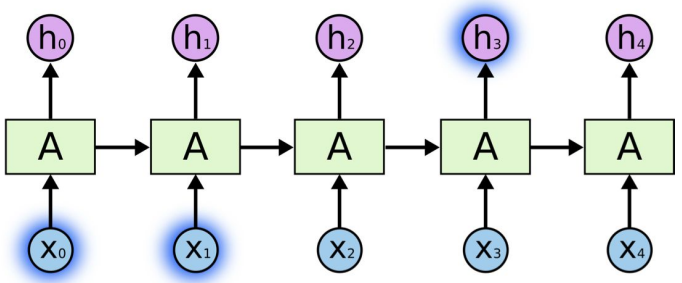
- For RNNs the ***tanh*** is to be preferred when mixing of the previous state and the new time input
  - **Output range: [-1, +1]** → useful to **add or subtract information from the memory state**
    - Sigmoid [0, +1] → only increasing values are allowed
    - ReLu [0, +∞] → only increasing values are allowed
  - **Zero centered** → helps to keep the gradients close to zero improving the speed of the convergence
  - **Stronger derivative wrt sigmoid** → signals survive longer during the temporal steps
    - Max derivative for sigmoid: 0.25
    - Max derivative for tanh: 1
  - Overall helps to **squash the result** of the linear combination of past state and present input to a "reasonable interval"
    - ReLu would make gradients explode because of the backprop through time

$$\tilde{h}_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

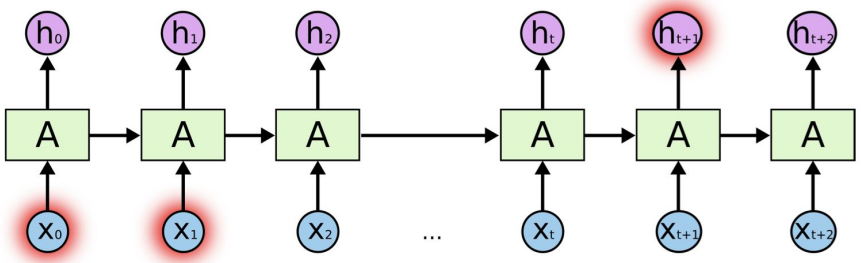
New state candidate

Problem with **long-term dependencies**

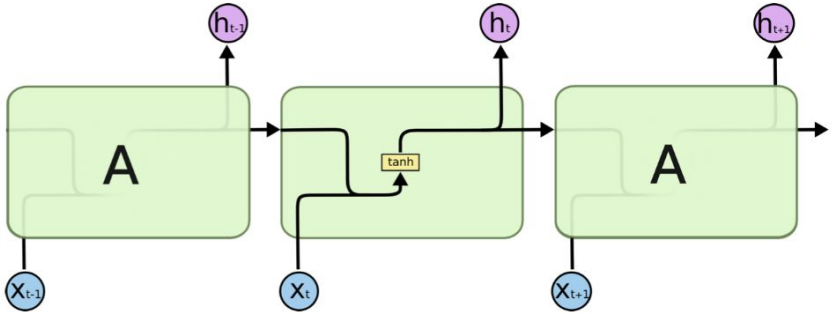
- For small gaps, simple RNNs can learn to use past information, e.g. predicting the last word in “the clouds are in the *sky*”



- As the gap grows, RNNs become unable to learn to connect the information in practice
  - Eventually the gradients go to zero
  - The memory state is being constantly overwritten by the new imput

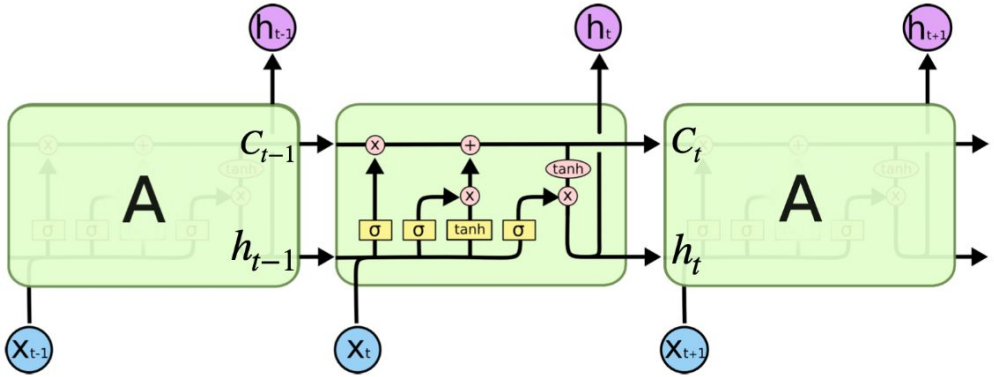


Solved with **Long Short-Term Memory** [ref.]



- in RNNs unbounded activations (like ReLU) cannot be used as they create instabilities
- tanh or sigmoid are OK but suffers vanishing of the gradient

problem solved in LSTM RNN (Hochreiter, '97) with a "software trick": uses four neural layers which interact in such a way to implement a sort of parallel data-flow which at each step t makes the information from previous steps available w/o being affected by gradient dilution

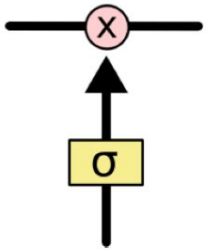


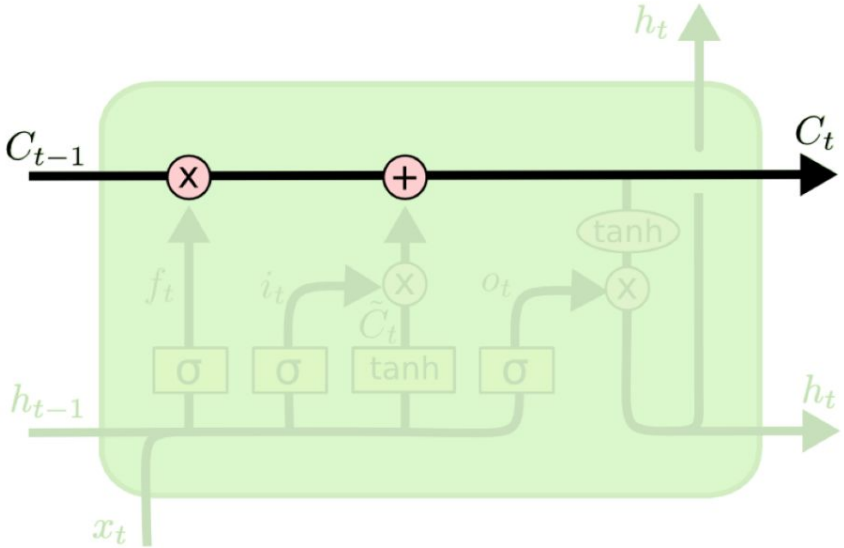
key element: cell-state C_t

is a memory units (“conveyor belt”) to which is possibile to add or subtract information using “gate” structures

Cell **state**

- Cell state is like a conveyer belt: runs straight down the entire chain, with only some minor linear interactions
- Gates optionally let information through: sigmoid outputs numbers between 0 and 1, describing how much should be let through



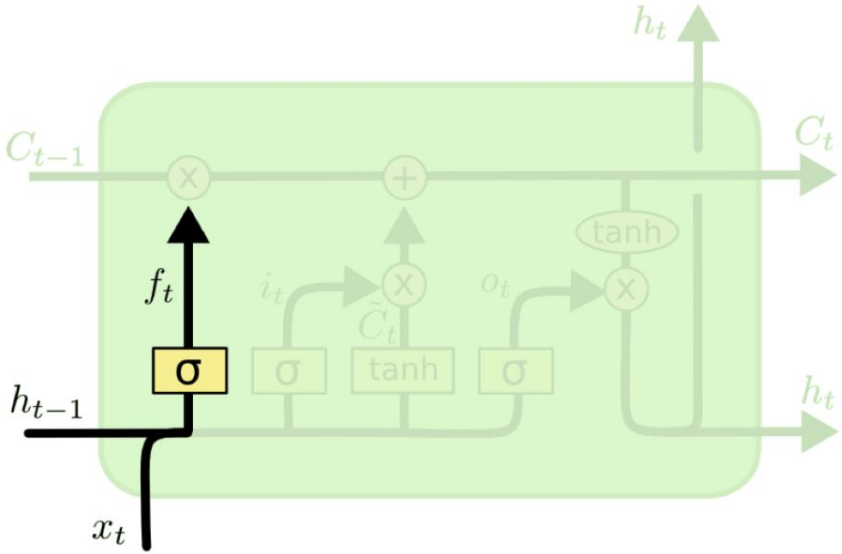


Forget gate layer

- Forget gate layer controls how much information to throw away from the cell state

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$$

0 → forget cell state
1 → preserve cell state

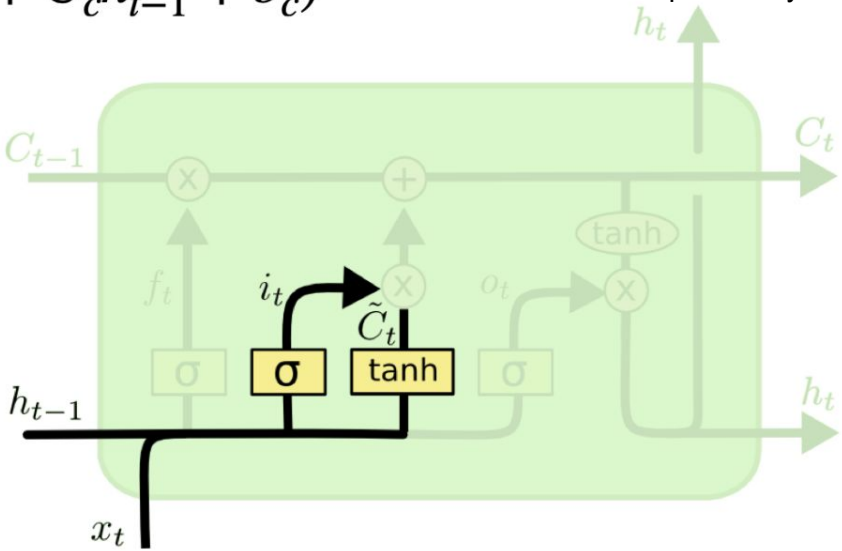


Input gate layer

- Input gate layer controls whether a new candidate value $\tilde{C}_t$ flows into the cell state

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$$ → what values to update

$$\tilde{C}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$$ → new candidates to be potentially added to the state

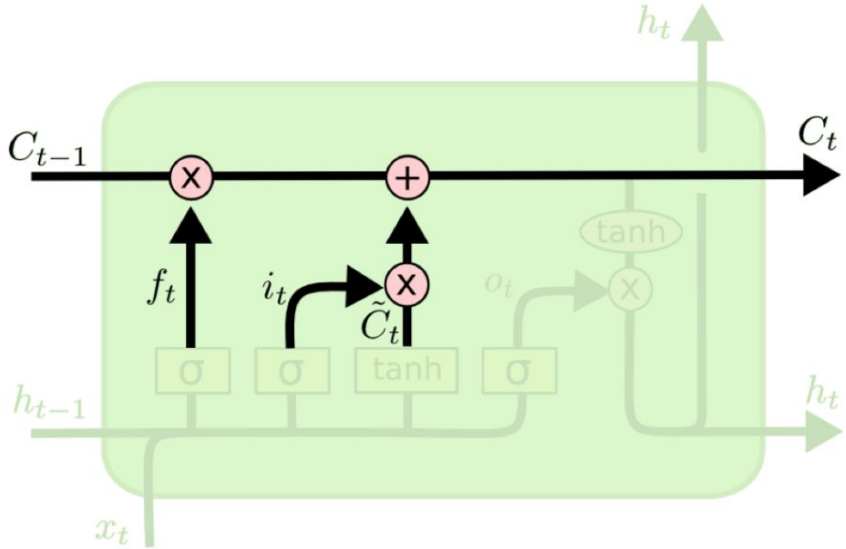


Cell **state update**

- Cell state is updated using forget and input gates

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$

→ thanks to the **sum**, the gradients flow backward without easily vanishing



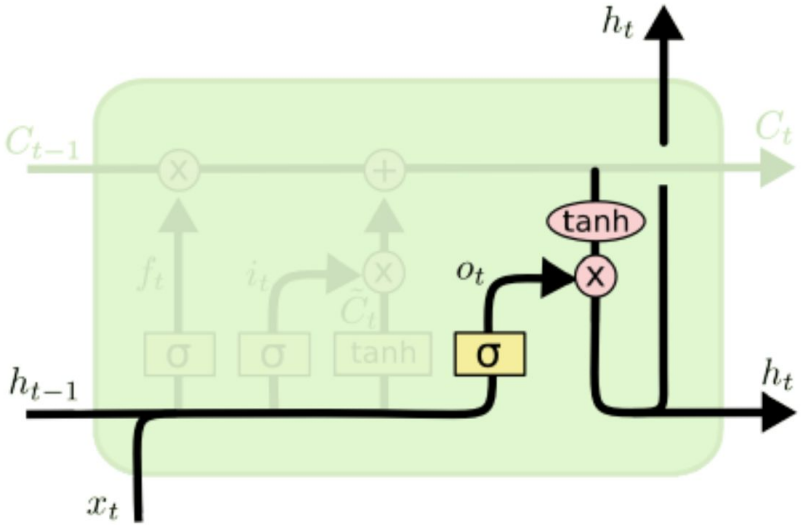
Output gate layer

- Finally, the output gate layer controls how the updated cell value contributes to the hidden state

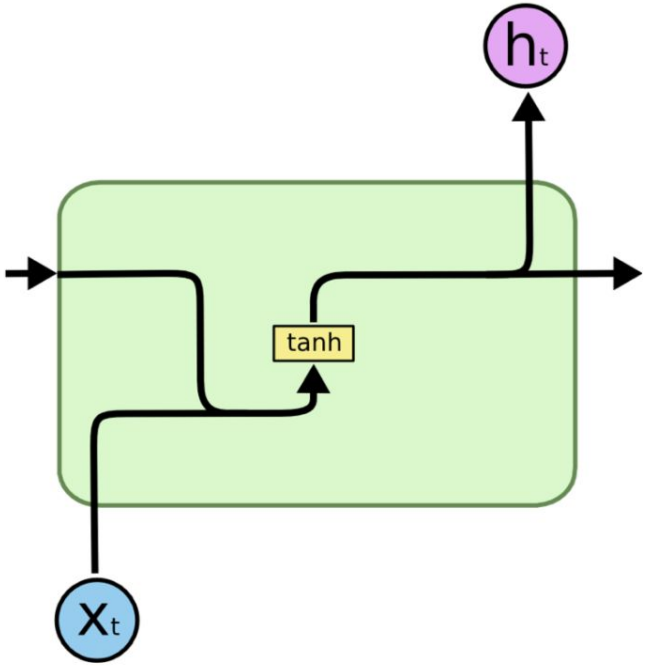
$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$$

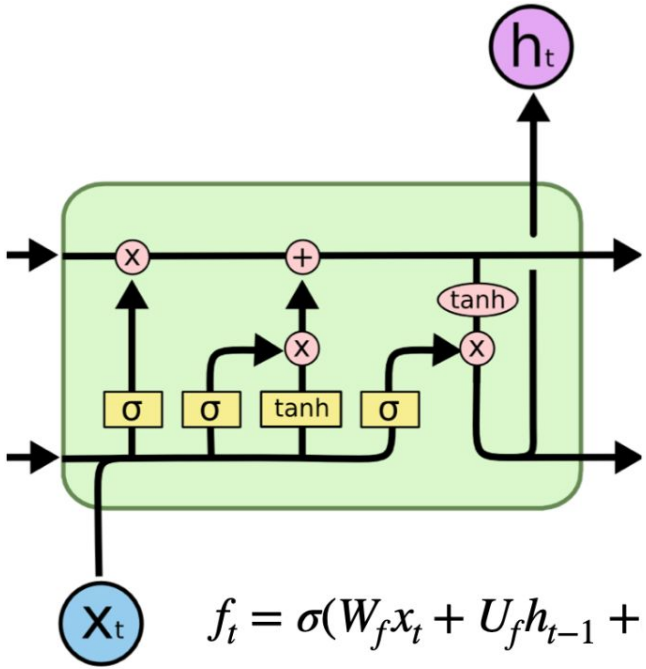
$$h_t = o_t \cdot \tanh(C_t)$$

→ filters out the memory to be sent to the next step



Simple RNN vs LSTM





$$h_t = \tanh(W_f x_t + U_f h_{t-1} + b_f)$$

$$f_t = \sigma(W_f x_t + U_f h_{t-1} + b_f)$$

$$i_t = \sigma(W_i x_t + U_i h_{t-1} + b_i)$$

$$\tilde{C}_t = \tanh(W_c x_t + U_c h_{t-1} + b_c)$$

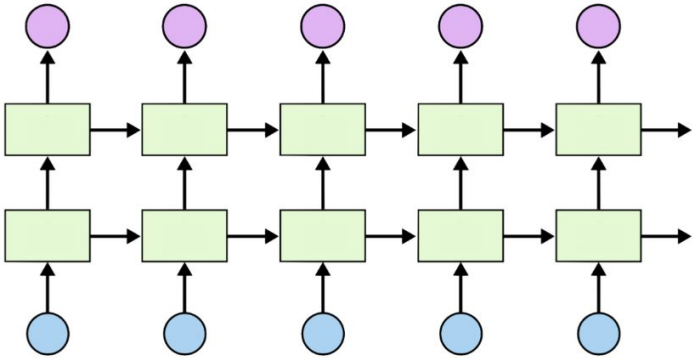
$$o_t = \sigma(W_o x_t + U_o h_{t-1} + b_o)$$

$$h_t = o_t \cdot \tanh(C_t)$$

Variations

Stacked RNNs

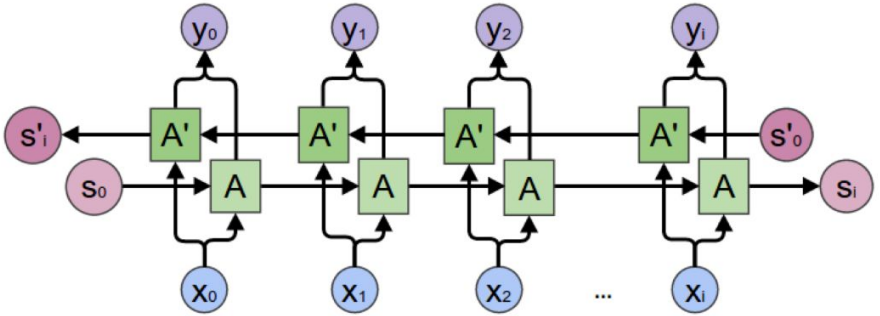
enables more complex representation
capturing information at different scales



```
# Initialize the RNN.
rnn = nn.RNN(input_size, hidden_size, num_layers = 2,
             nonlinearity='tanh', batch_first=True)
```

Bidirectional RNNs

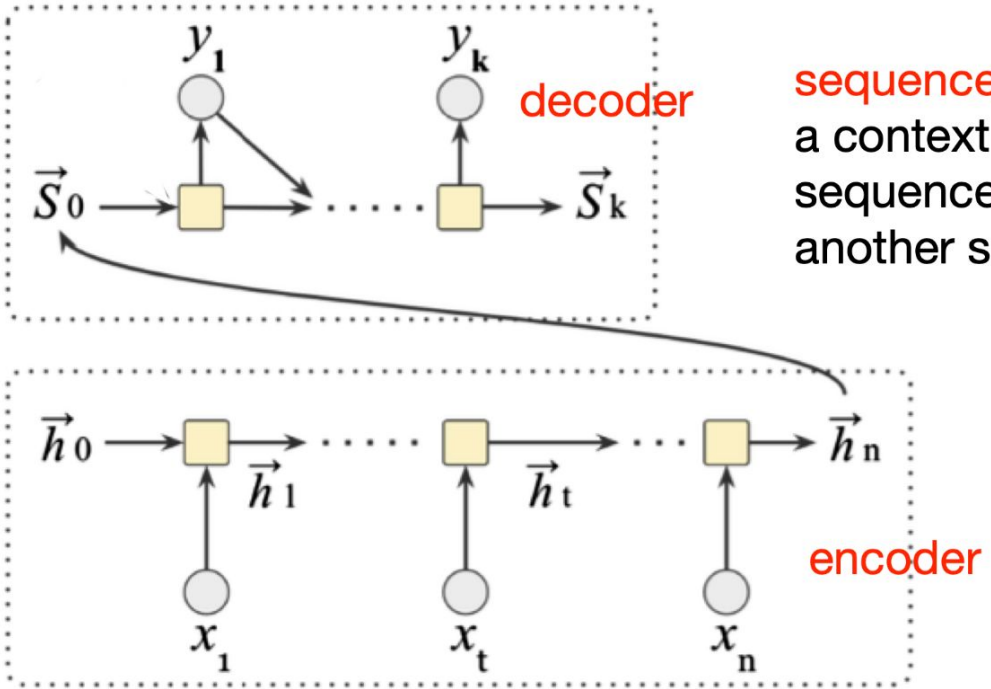
inputs are processed in both forward and reverse time order
allows a BRNN to look at future context as well




```
# Initialize the RNN.
rnn = nn.RNN(input_size, hidden_size, num_layers = 1,
             nonlinearity='tanh',
             bidirectional = True, batch_first=True)
```

Variations

Encoder-Decoder RNNs



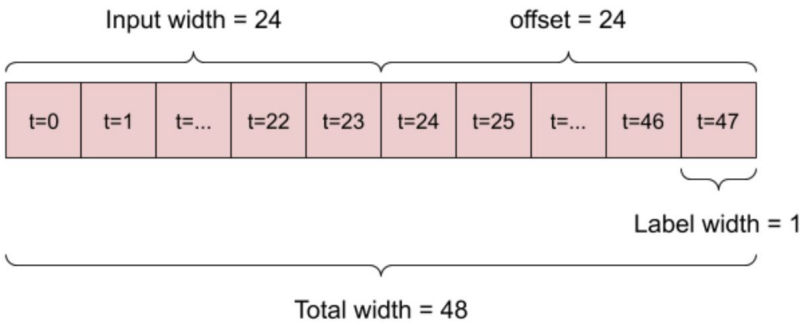
sequence2sequence: uses an RNN to generate a context vector that encode the whole input sequence and a separate RNN to generate another sequence from the encoded context

often used in machine translation

How to handle **continuous data**: data windowing

- The models make a set of predictions based on a window of consecutive samples from the data

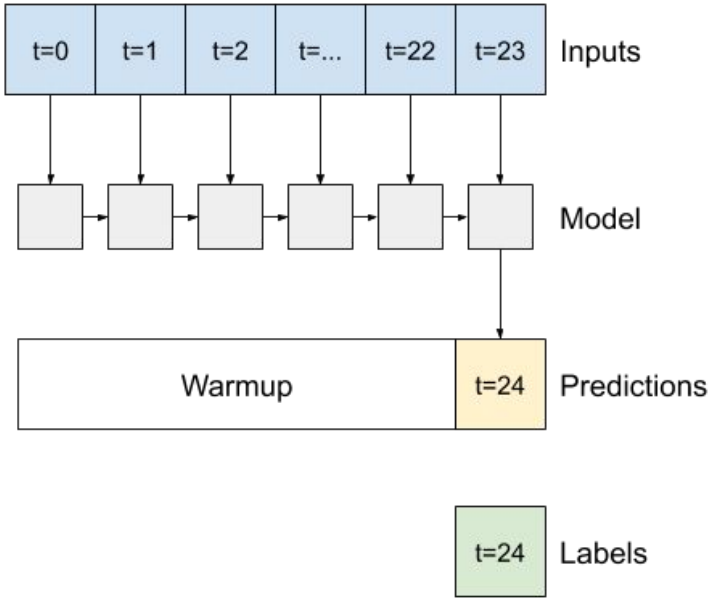
- The main features of the input windows are:
  - **The width** (number of time steps) of the input and label windows
  - **The time offset** between them
  - **Which features** are used as inputs, labels, or both



Example: single prediction 24 hours into the future, given 24 hours of history

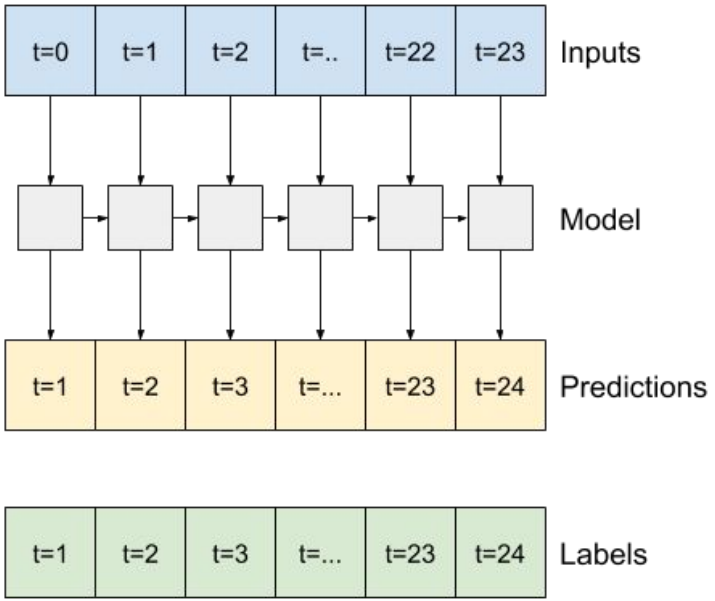
LSTM `return_sequences`

False



The layer only returns the output of the final time step, giving the model time to warm up its internal state before making a single prediction → **only the final verdict is returned**

True



The layer returns an output for each input. This is useful for stacking RNN layers as **the following layer needs data distributed in time**

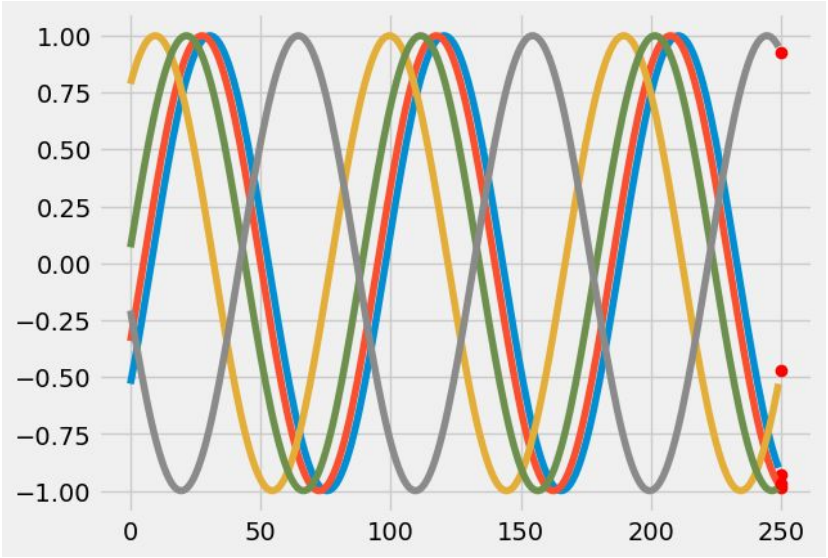
Example 1:
Synthetic **sinusoidal dataset**

Predict patterns on a synthetic 1D dataset (Recursive Forecasting)

- Let's use a **sinusoid function**. The example can be accessed here.
  - As usual copy it to your area if you want to modify it

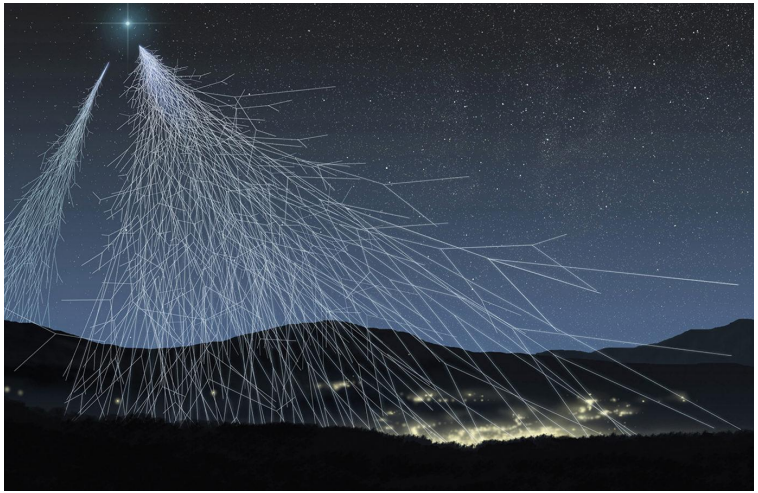
- Basic idea:
  - Start from a sine dataset
  - Train a LSTM and make it learn the pattern
  - **Predict 5 periods in the future**
  - **Autoregressive approach**

- Comments: does it work well and why?



Example 2:
Classify **cosmic ray radio signals**

Cosmic ray **radio signals**



- The accurate detection of radio emission can be key to **determining the properties of cosmic-ray showers**, such as the energy and depth of the electromagnetic component

- Particle showers are created when cosmic rays interact with the atmosphere
  - Electrons and positrons are accelerated in opposite directions by the Earth magnetic field → electrical current with associated **EM pulse**
  - Very **faint radio signals** are created
  - Antennas and filters have a finite bandwidth → oscillating signal

- Signals are **drowned out by bkg** from universe and man-made sources
  - Crucial to find a way to identify them in the noise



https://arxiv.org/pdf/1901.04079
https://arxiv.org/pdf/2508.14711

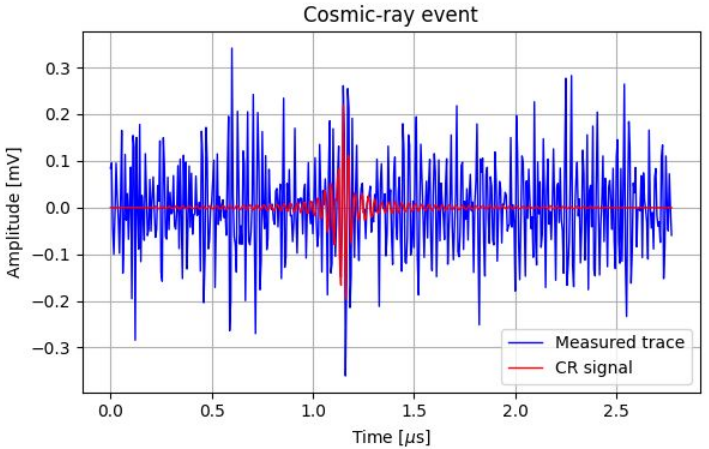
Classify cosmic ray radio signals

- Goal is to **identify radio signals** emitted by ultra-high energy cosmic rays (UHECRs) that initiate extensive particle showers in the atmosphere
  - measuring pulses of a few nanoseconds in length emitted by cosmic ray air showers with energies above 10^{17} eV

- Simulated data produced with **SNR values between 0.5 and 5.0**
  - SNR ~ 1.5 shown
  - Preprocessing applied: normalize each generated time trace by dividing each bin by the standard deviation of the whole trace

$$\mathrm{SNR} = \frac{\text{max signal}}{\mathrm{RMS}_{\text{noise}}} = \frac{A^S_{\max}}{\sqrt{\frac{1}{N}\sum_{i=1}^{N} A_i^2}}$$

- [Classify signal vs bkg only events with LSTMs]



Exercise 1:
Build on the **radio signals** example

1. Replace the LSTM layers with **bidirectional LSTM layers**: `layers.Bidirectional(layers.LSTM(...))` (See https://keras.io/api/layers/recurrent_layers/bidirectional/).
    - Note for the first layer, the `input_shape` argument needs to be an input to the outer `layers.Bidirectional` function instead of the inner `layers.LSTM` function.
    - How does the number of parameters of the model change? How does the performance change?

2. Replace the LSTM layers with **1D convolutional layers** `layers.Conv1D(...)` with the following parameters.
    - `filters=32, kernel_size=5, strides=1, padding="same", activation="relu"`
    - `filters=64, kernel_size=5, strides=2, padding="same", activation="relu"`
    - `filters=10, kernel_size=5, strides=2, padding="same", activation-"relu"`
    - How does the number of parameters of the model change? How does the performance change?

3. How can you modify the architecture and data to perform signal **trace extraction (de-noising)**?

Exercise 2:
Weather forecast with LSTM

Use **real data** to train a LSTM net and make predictions

- A dataset containing the following features is made available here:
  - **Temperature, wind speed, wind direction, global radiation, rain, humidity**
- The dataset has been downloaded from the italian ARPA website and contains data from the **early 2000s to April 2025** as registered by a weather station in Lambrate (MI)
  - See the form to request the dataset here:
    - https://www.arpalombardia.it/temi-ambientali/aria/form-richiesta-dati-stazioni-fisse/
    - https://www.arpalombardia.it/temi-ambientali/meteo-e-clima/form-richiesta-dati/
- The goal is to **build and train a model and perform predictions on one or multiple features of the dataset**
  - E.g.: predict the temperature hour-by-hour in the next 24h
- You have the **freedom to pick a different dataset** and build a net to perform a similar task

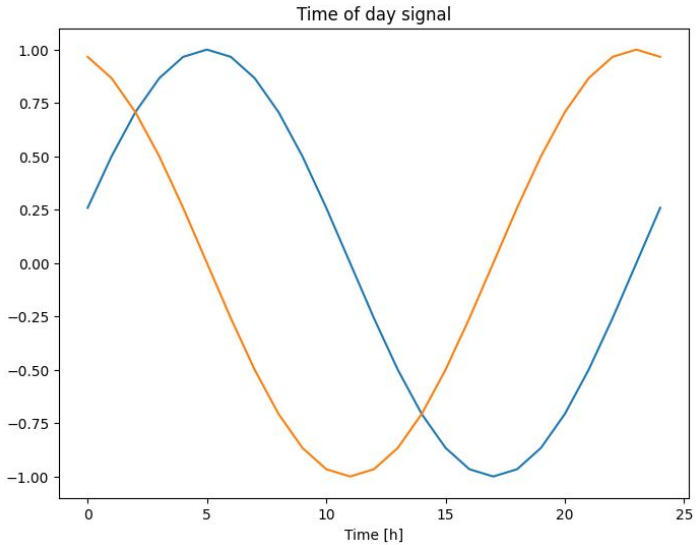
Feature engineering and suggestions

- As always, before diving into the model construction, it is important to spend some time trying to **get the maximum out of the data**
  - In addition to perform **data normalization** and some **data cleaning**, some features can be manipulated

- Example: date and time could be very useful but not if they are in string format

```
day = 24*60*60
year = (365.2425)*day

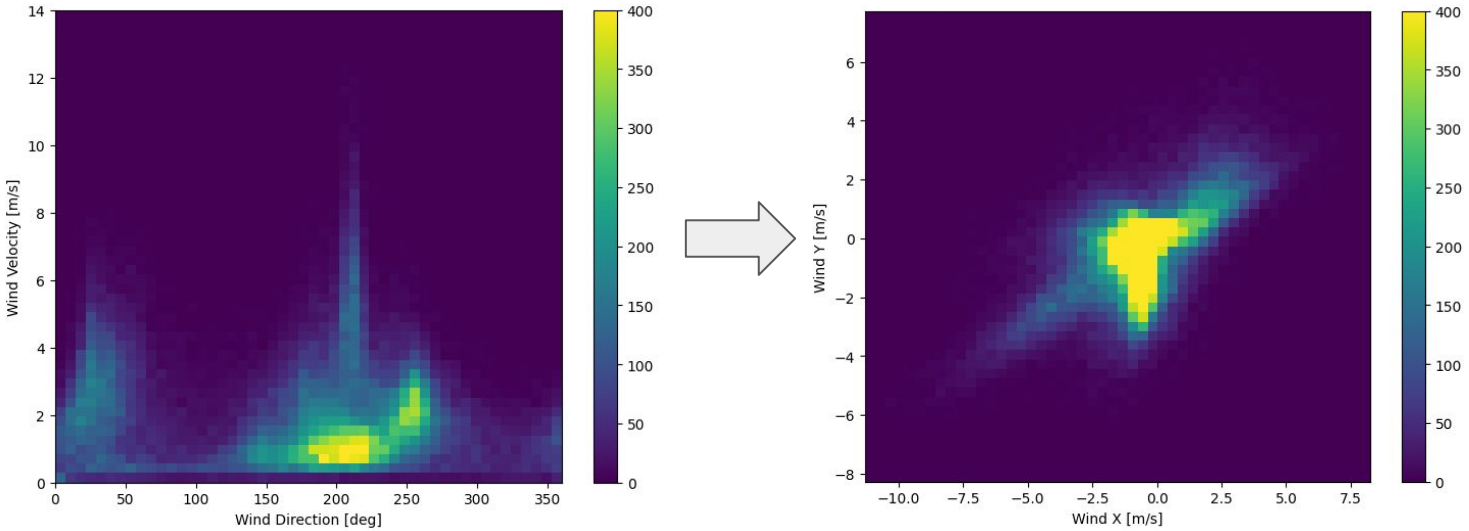
df['Day sin'] = np.sin(timestamp_s * (2 * np.pi / day))
df['Day cos'] = np.cos(timestamp_s * (2 * np.pi / day))
df['Year sin'] = np.sin(timestamp_s * (2 * np.pi / year))
df['Year cos'] = np.cos(timestamp_s * (2 * np.pi / year))
```



Feature engineering and suggestions

- As always, before diving into the model construction, it is important to spend some time trying to **get the maximum out of the data**
  - In addition to perform **data normalization** and some **data cleaning**, some features can be manipulated

- Example:
  - Wind direction is in degrees [0°, 360°] → need some wrapping
  - Also the wind direction shouldn't matter if the wind speed is 0



Quantify performance

- It is always important to compare the results obtained against what a **baseline model** returns
  - Need to decide what a good and stable reference model could be

- In this case the simplest reference model could be the one that replicates the last reading

